Final project: Mountains vs. Beaches Preferences

CS505 - Data Mining

Group : FP L

Name : Ahmad Issa

Student Number:002230777

Name: Jibril Hussaini

Student Number: 002354055

Load the Dataset:

Age	Gender	Income	Education_Level	Travel_Frequency	Preferred_Activities	Vacation_Budget	Location	Proximity_to_Mountains	Proximity_to_Beaches	Favorite_Season	Pets	Environmental_Concerns
56	male	71477	bachelor	9	skiing	2477	urban	175	267	summer	0	1
69	male	88740	master	1	swimming	4777	suburban	228	190	fall	0	1
46	female	46562	master	0	skiing	1469	urban	71	280	winter	0	0
32	non-binary	99044	high school	6	hiking	1482	rural	31	255	summer	1	0
60	female	106583	high school	5	sunbathing	516	suburban	23	151	winter	1	1

Vacation_Budget	Location	Proximity_to_Mountains	Proximity_to_Beaches	Favorite_Season	Pets	Environmental_Concerns	Preference
2477	urban	175	267	summer	0	1	1
4777	suburban	228	190	fall	0	1	0
1469	urban	71	280	winter	0	0	1
1482	rural	31	255	summer	1	0	1
516	suburban	23	151	winter	1	1	0

Data Preprocessing

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52444 entries, 0 to 52443
Data columns (total 14 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Age                    52444 non-null  int64
 1   Gender                 52444 non-null  object
 2   Income                 52444 non-null  int64
 3   Education_Level        52444 non-null  object
 4   Travel_Frequency       52444 non-null  int64
 5   Preferred_Activities   52444 non-null  object
 6   Vacation_Budget        52444 non-null  int64
 7   Location               52444 non-null  object
 8   Proximity_to_Mountains 52444 non-null  int64
 9   Proximity_to_Beaches   52444 non-null  int64
 10  Favorite_Season        52444 non-null  object
 11  Pets                   52444 non-null  int64
 12  Environmental_Concerns 52444 non-null  int64
 13  Preference             52444 non-null  int64
dtypes: int64(9), object(5)
memory usage: 5.6+ MB
```

Display dataset statistic and summary

```
Dataset Summary:
                Age        Gender         Income  Education_Level  \
count  52444.000000  52444.000000   52444.000000     52444.000000
mean      43.507360      0.993250   70017.271280         1.492888
std       14.985597      0.816001   28847.560428         1.115580
min       18.000000      0.000000   20001.000000         0.000000
25%       31.000000      0.000000   45048.250000         0.000000
50%       43.000000      1.000000   70167.000000         1.000000
75%       56.000000      2.000000   95108.500000         2.000000
max       69.000000      2.000000  119999.000000         3.000000

       Travel_Frequency  Preferred_Activities  Vacation_Budget      Location  \
count      52444.000000          52444.000000     52444.000000  52444.000000
mean           4.489265              1.496282      2741.799062      1.000210
std            2.876130              1.115204      1296.922423      0.816251
min            0.000000              0.000000       500.000000      0.000000
25%            2.000000              1.000000      1622.000000      0.000000
50%            4.000000              1.000000      2733.000000      1.000000
75%            7.000000              2.000000      3869.000000      2.000000
max            9.000000              3.000000      4999.000000      2.000000

       Proximity_to_Mountains  Proximity_to_Beaches  Favorite_Season  \
count            52444.000000          52444.000000     52444.000000
mean               149.943502            149.888452         1.499028
std                 86.548644             86.469248         1.117481
min                  0.000000              0.000000         0.000000
25%                 75.000000             75.750000         0.000000
50%                150.000000            150.000000         2.000000
75%                225.000000            225.000000         2.000000
max                299.000000            299.000000         3.000000

              Pets  Environmental_Concerns    Preference
count  52444.000000            52444.000000  52444.000000
mean       0.500858                0.498436      0.250706
std        0.500004                0.500002      0.433423
min        0.000000                0.000000      0.000000
25%        0.000000                0.000000      0.000000
50%        1.000000                0.000000      0.000000
75%        1.000000                1.000000      1.000000
max        1.000000                1.000000      1.000000
```

Check For Missing Values

```
Missing Values in Each Column:
Age                          0
Gender                       0
Income                       0
Education_Level              0
Travel_Frequency             0
Preferred_Activities         0
Vacation_Budget              0
Location                     0
Proximity_to_Mountains       0
Proximity_to_Beaches         0
Favorite_Season              0
Pets                         0
Environmental_Concerns       0
Preference                   0
dtype: int64
```

Distribution of Target Variable (Preference):

```
Distribution of Target Variable (Preference):
Preference
0    39296
1    13148
Name: count, dtype: int64
```

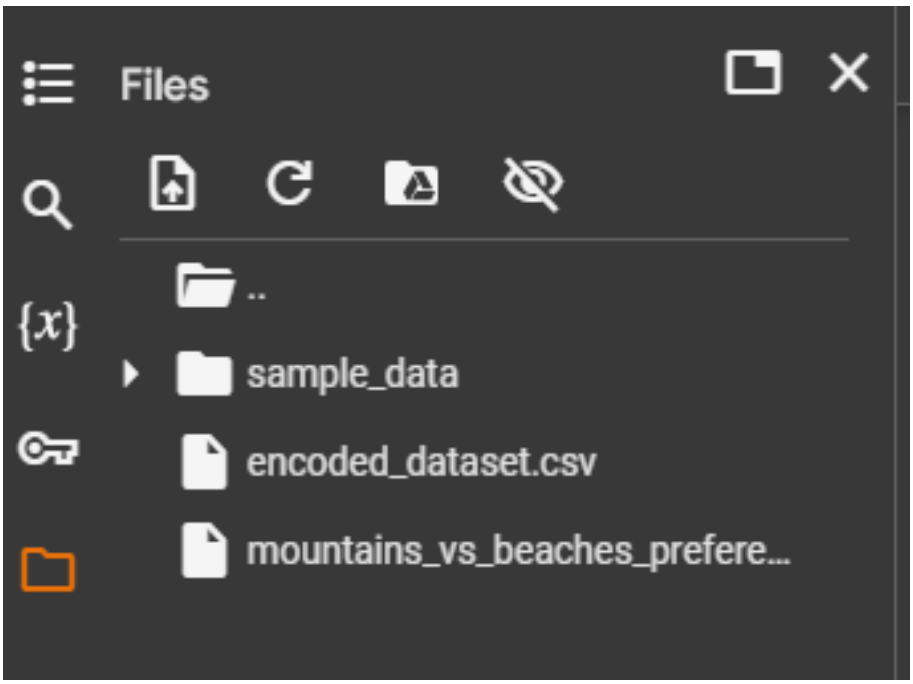
Turning Data into Numerical values

The dataset was transformed into numerical values using a combination of label encoding and one-hot encoding techniques.

1. Label Encoding: The Education_Level column, which had ordinal categorical data, was label encoded. This converted its unique string values into corresponding integer representations while preserving the ordinal relationship.
2. One-Hot Encoding: Remaining categorical columns (Gender, Preferred_Activities, Location, Favorite_Season) were one-hot encoded. This approach created binary indicator variables for each unique category, ensuring a non-ordinal representation of these features.
3. Output: After encoding, the dataset expanded from its original shape of (52444, 14) to (52444, 20), reflecting the additional binary columns generated.

A snapshot of the transformed dataset shows the successful application of encoding techniques, with both label and one-hot encoded features integrated seamlessly.

Stored The Encoded Dataset



Dataset after Encoding (Education_Level Label Encoded, Others One-Hot Encoded):

	Age	Income	Education_Level	Travel_Frequency	Vacation_Budget	Proximity_to_Mountains	Proximity_to_Beaches	Pets	Environmental_Concerns	Preference	Gender_male	Gender_non-binary	Preferred_Activities_skiing	Preferred_Activities_sunbathing	Preferred_Activities_swimming	Location_suburban	Location_urban	Favorite_Season_spring	Favorite_Season_summer	Favorite_Season_winter
0	56	71477	0	9	2477	175	267	0	1	1	1	0	1	0	0	0	1	0	1	0
1	69	88740	3	1	4777	228	190	0	1	0	1	0	0	0	1	1	0	0	0	0
2	46	46562	3	0	1469	71	280	0	0	1	0	0	1	0	0	0	1	0	0	1
3	32	99044	2	6	1482	31	255	1	0	1	0	1	0	0	0	0	0	0	1	0
4	60	106583	2	5	516	23	151	1	1	0	0	0	0	1	0	1	0	0	0	1

Shape of the dataset before encoding: (52444, 14)
Shape of the dataset after encoding: (52444, 20)

Following the encoding process, the transformed dataset was split into a matrix of features (X) and a target vector (y):

1. Matrix of Features (X):
    - The Preference column, which represents the target variable, was excluded to define the feature matrix X.
    - This matrix comprises 19 features, including both numerical and encoded categorical variables.
    - The shape of X is (52444, 19).
2. Target Vector (y):
    - The Preference column was extracted to define the target vector y.
    - It contains binary labels representing the preference classification for each sample.
    - The shape of y is (52444,).
3. Summary:
    - The prepared feature matrix and target vector serve as the input data for subsequent modeling tasks.
    - A preview of X and y confirms the successful partitioning of the dataset into features and labels.

```
Shape of Feature Matrix (X): (52444, 19)
Shape of Target Vector (y): (52444,)

Feature Matrix (X):
   Age  Income  Education_Level  Travel_Frequency  Vacation_Budget  \
0   56   71477                0                 9             2477
1   69   88740                3                 1             4777
2   46   46562                3                 0             1469
3   32   99044                2                 6             1482
4   60  106583                2                 5              516

   Proximity_to_Mountains  Proximity_to_Beaches  Pets  Environmental_Concerns  \
0                     175                   267     0                       1
1                     228                   190     0                       1
2                      71                   280     0                       0
3                      31                   255     1                       0
4                      23                   151     1                       1

   Gender_male  Gender_non-binary  Preferred_Activities_skiing  \
0            1                  0                            1
1            1                  0                            0
2            0                  0                            1
3            0                  1                            0
4            0                  0                            0
```

```
   Preferred_Activities_sunbathing  Preferred_Activities_swimming  \
0                                0                             0
1                                0                             1
2                                0                             0
3                                0                             0
4                                1                             0

   Location_suburban  Location_urban  Favorite_Season_spring  \
0                  0               1                       0
1                  1               0                       0
2                  0               1                       0
3                  0               0                       0
4                  1               0                       0

   Favorite_Season_summer  Favorite_Season_winter
0                       1                       0
1                       0                       0
2                       0                       1
3                       1                       0
4                       0                       1

Target Vector (y):
0    1
1    0
2    1
3    1
4    0
Name: Preference, dtype: int64
```

Exploratory Data Analysis:

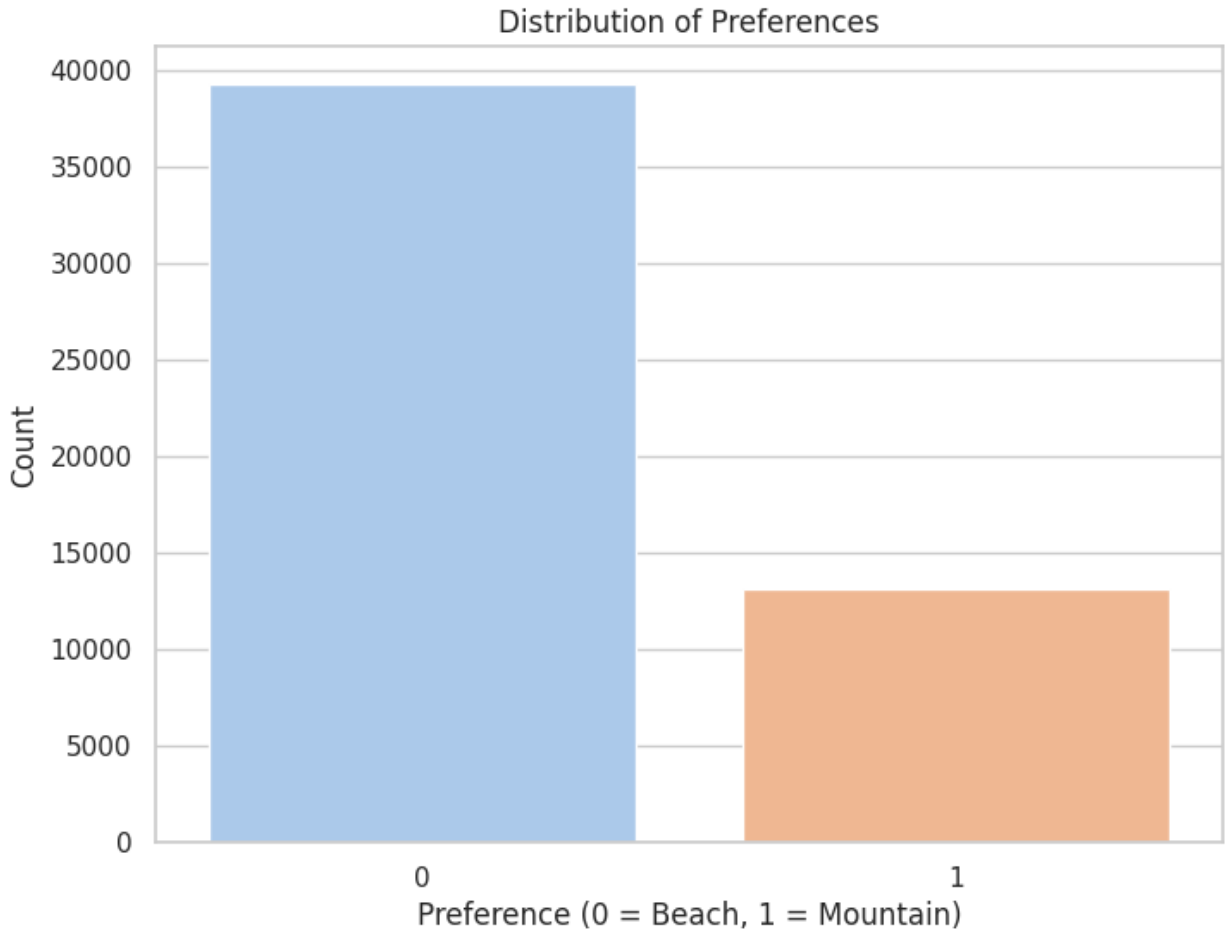
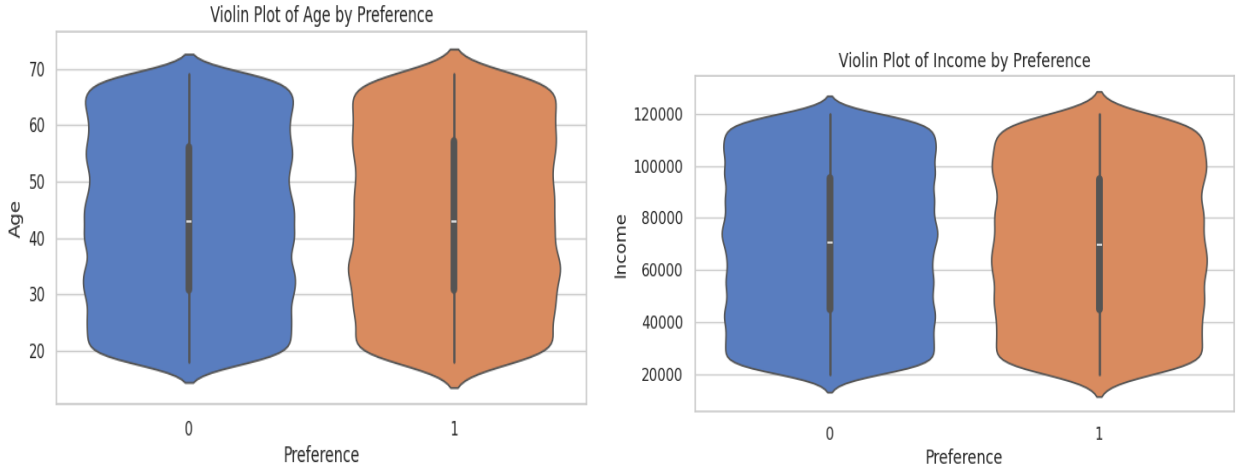
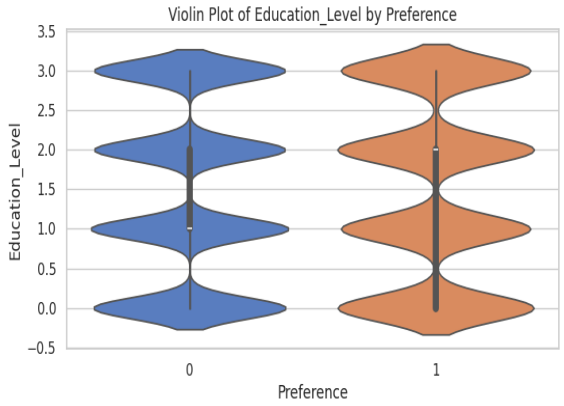
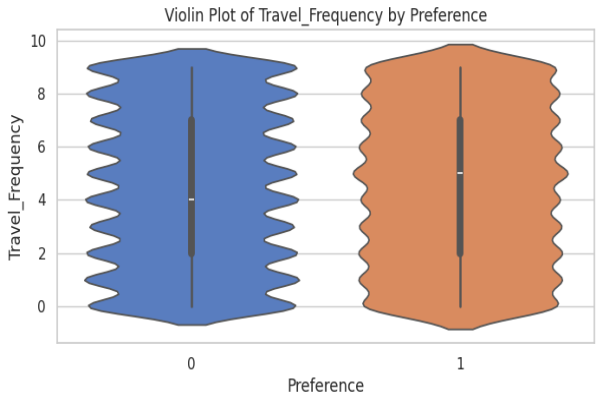


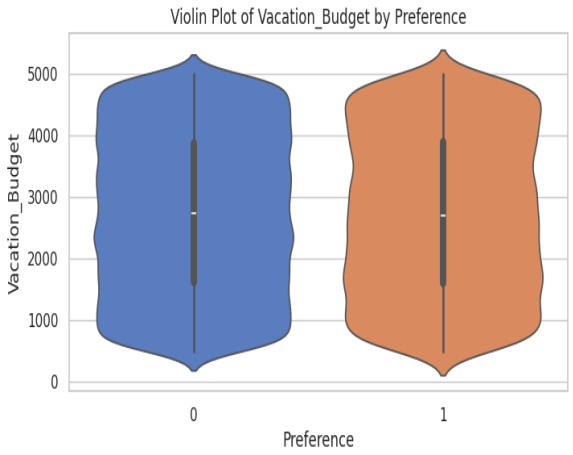
Figure 1: Distribution of Preferences

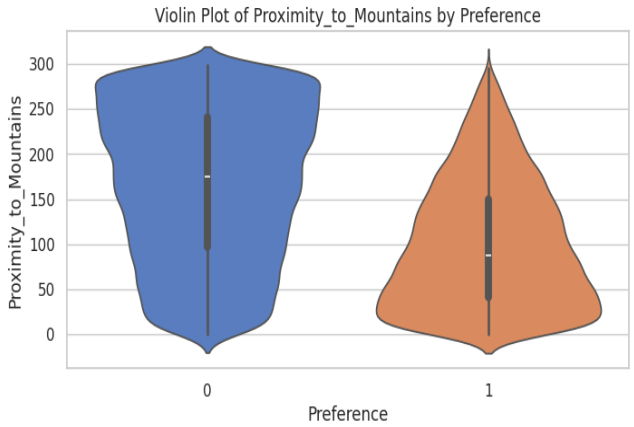
Figure 2: Histogram for Numeric Features

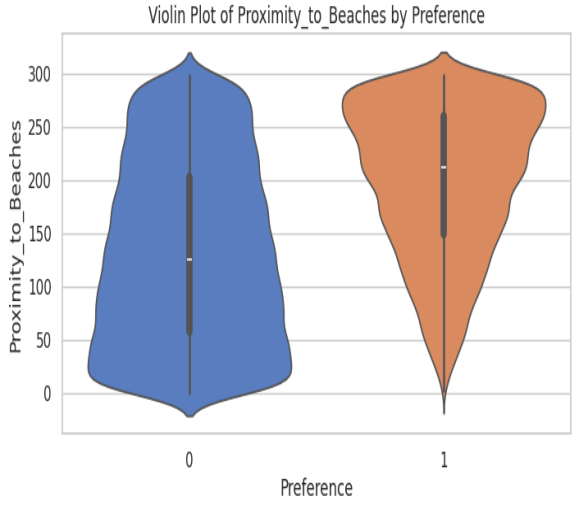


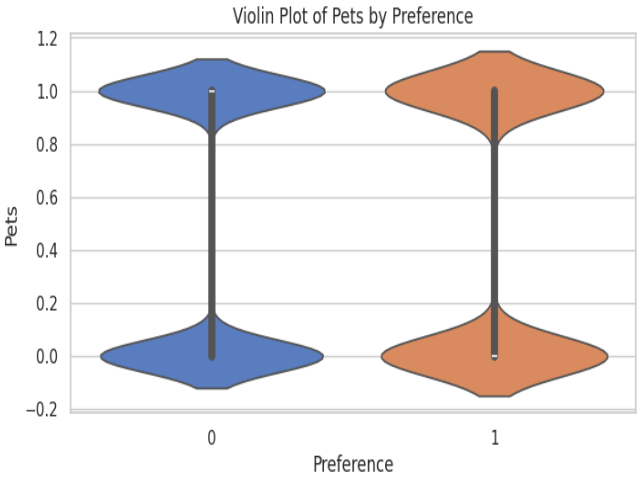

Violin Plot of Education_Level by Preference
Education_Level
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−0.5
0
1
Preference

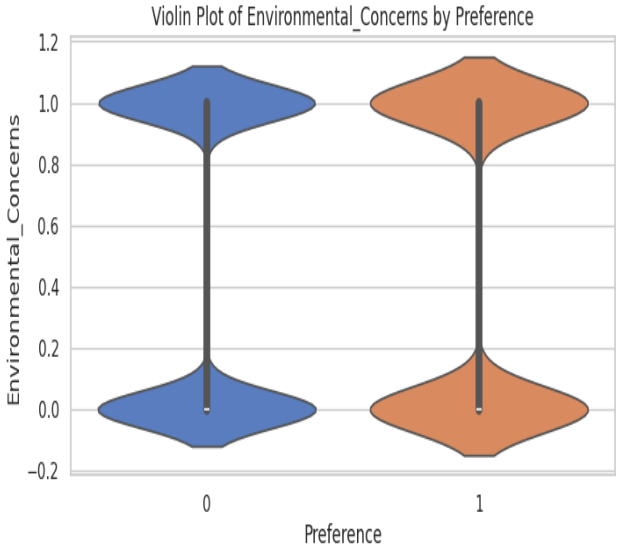

Violin Plot of Travel_Frequency by Preference
Travel_Frequency
10
8
6
4
2
0
0
1
Preference

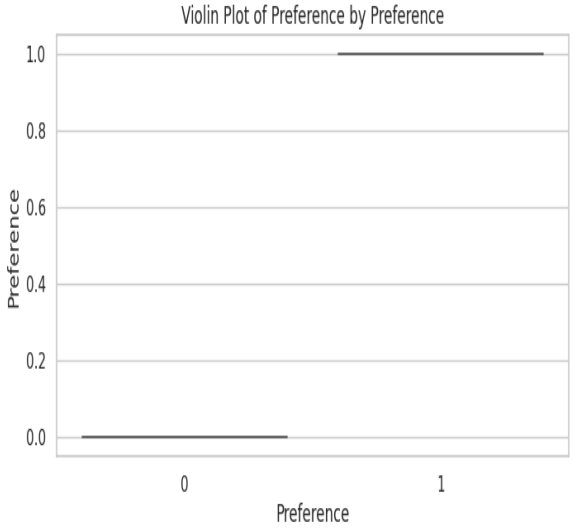

Violin Plot of Vacation_Budget by Preference
Vacation_Budget
5000
4000
3000
2000
1000
0
0
1
Preference

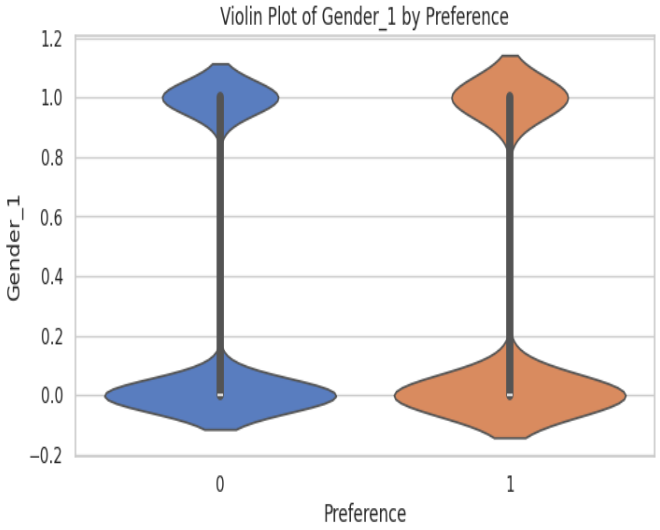

Violin Plot of Proximity_to_Mountains by Preference
Proximity_to_Mountains
300
250
200
150
100
50
0
0
1
Preference

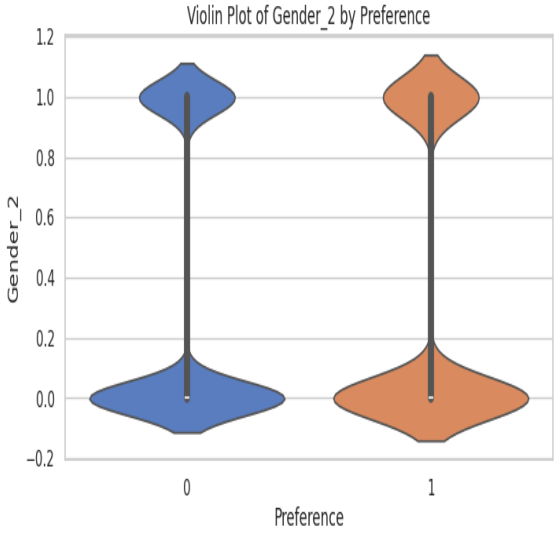

Violin Plot of Proximity_to_Beaches by Preference
Proximity_to_Beaches
300
250
200
150
100
50
0
0
1
Preference

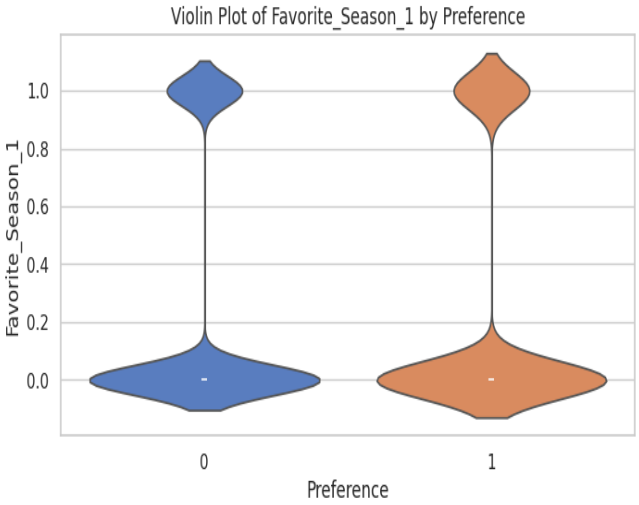

Violin Plot of Pets by Preference
Pets
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
0
1
Preference

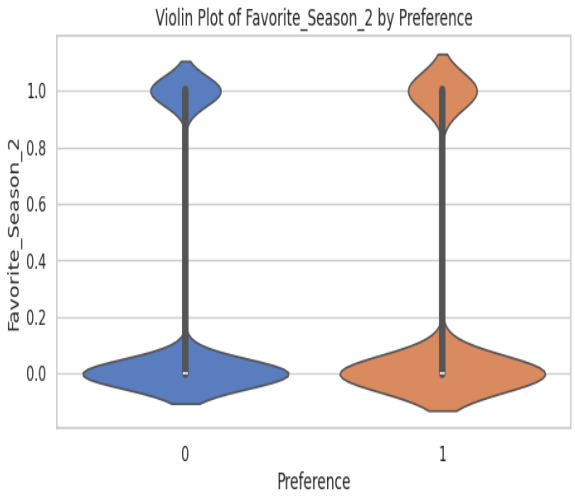

Violin Plot of Environmental_Concerns by Preference
Environmental_Concerns
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
0
1
Preference

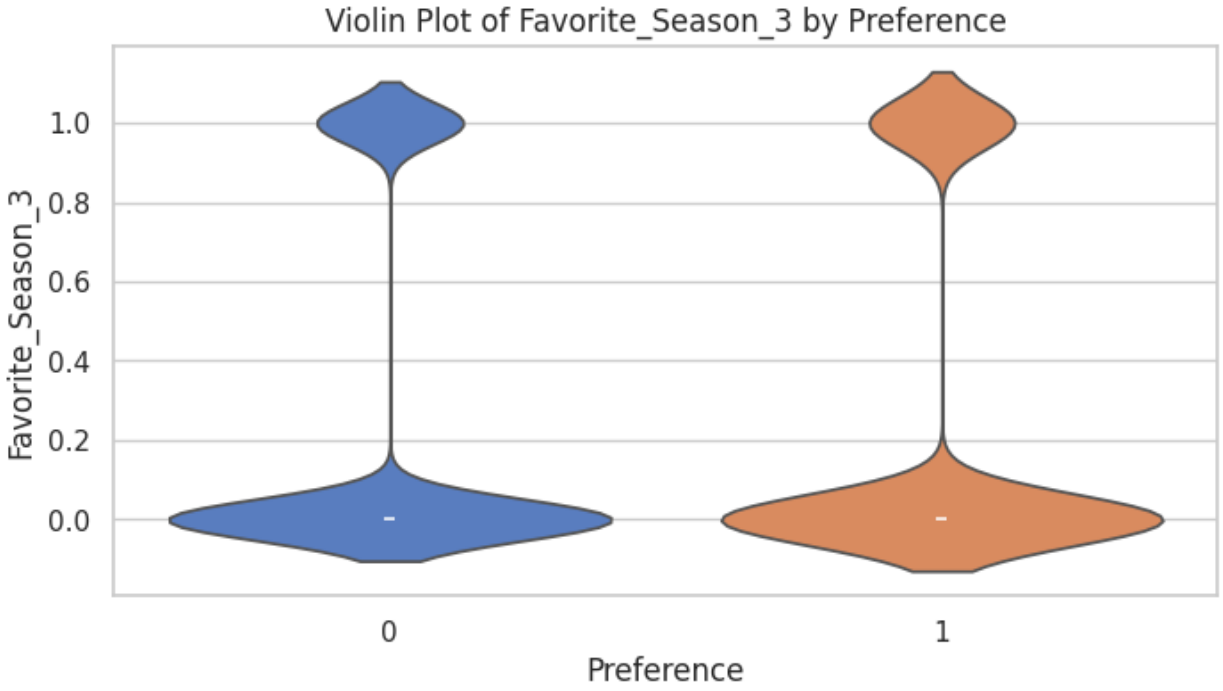

Violin Plot of Preference by Preference
Preference
1.0
0.8
0.6
0.4
0.2
0.0
0
1
Preference

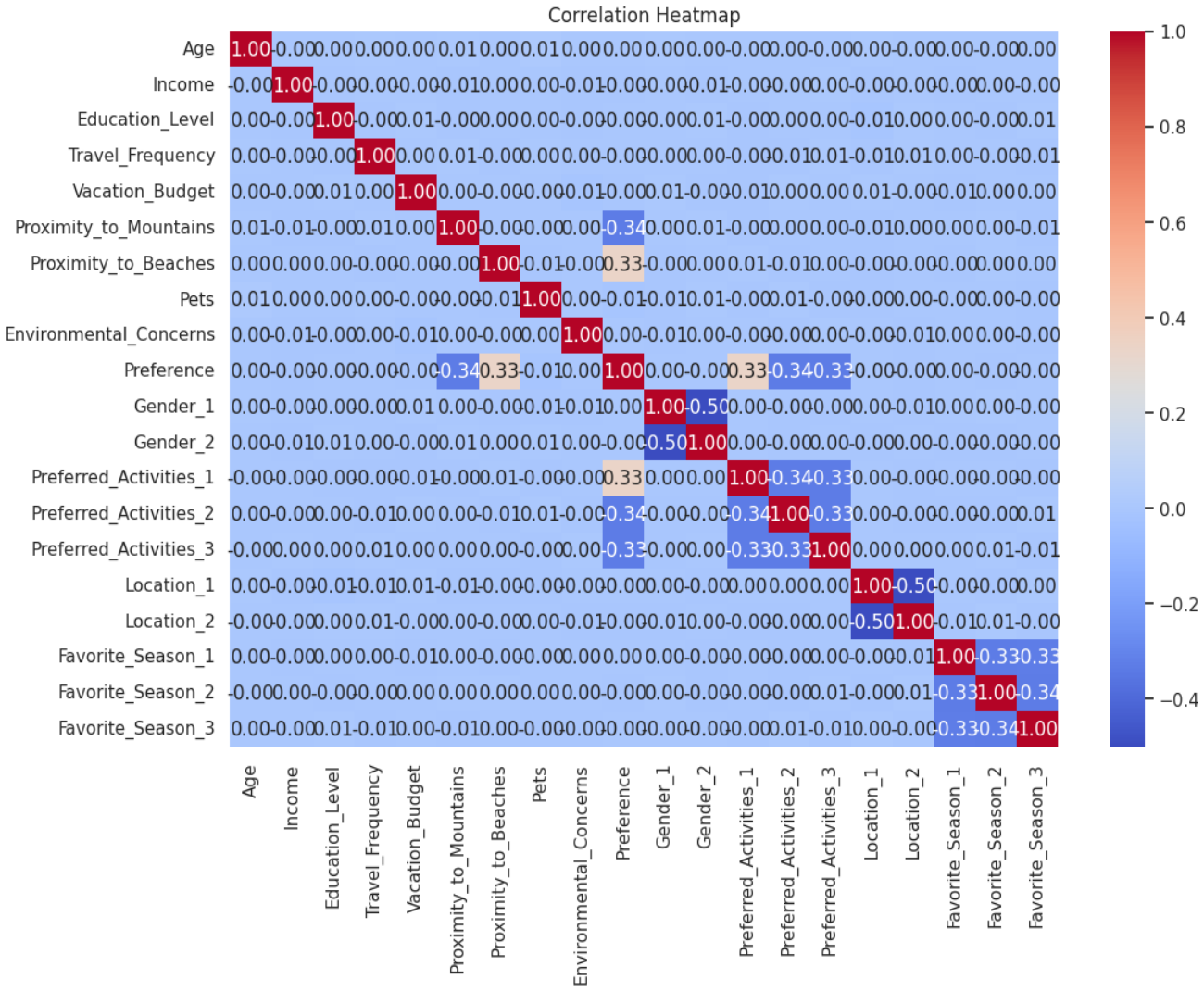

Violin Plot of Gender_1 by Preference
Gender_1
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
0
1
Preference


Violin Plot of Gender_2 by Preference
Gender_2
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
0
1
Preference


Violin Plot of Favorite_Season_1 by Preference
Favorite_Season_1
1.0
0.8
0.6
0.4
0.2
0.0
0
1
Preference


Violin Plot of Favorite_Season_2 by Preference
Favorite_Season_2
1.0
0.8
0.6
0.4
0.2
0.0
0
1
Preference


Violin Plot of Favorite_Season_3 by Preference
Favorite_Season_3
1.0
0.8
0.6
0.4
0.2
0.0
0
1
Preference



Data Spliting 80/20:

To enhance model training performance, the **SMOTE (Synthetic Minority Oversampling Technique)** was applied to address class imbalance in the training data.

1. **Original Data Split**:
    - The dataset was initially split into training and testing subsets:
        - **Training Feature Matrix (X_train):** (41955, 19)
        - **Testing Feature Matrix (X_test):** (10489, 19)
        - **Training Target Vector (y_train):** (41955,)
        - **Testing Target Vector (y_test):** (10489,)
    - The training data exhibited significant class imbalance, as shown below:

    - Class 0: 31437 samples
    - Class 1: 10518 samples

2. **Post-SMOTE Balancing**:
    - SMOTE generated synthetic samples to balance the minority class (1) to match the majority class (0), resulting in the following balanced distribution:
        - Class 0: 31437 samples
        - Class 1: 31437 samples
    - The balanced training data shapes were updated as follows:
        - **X_train_balanced:** (62874, 19)
        - **y_train_balanced:** (62874,)
3. **Testing Data**:
    - The testing data was left unchanged to ensure unbiased model evaluation:
        - **X_test:** (10489, 19)
        - **y_test:** (10489,)
4. **Conclusion**:
    - By balancing the training data with SMOTE, the model training process is expected to improve, as it mitigates biases caused by class imbalance.

Data was Standarized:

```
Scaled Training Data:
        Age    Income  Education_Level  Travel_Frequency  Vacation_Budget  \
0 -1.701142  0.051251          1.35089         -1.554597        -1.387448
1  1.101772  0.537643         -0.44117          0.878933         1.183821
2  0.167468  1.464563         -0.44117         -0.859303         1.075078
3  0.167468  0.276385         -1.33720         -0.164009        -1.179988
4 -1.100517 -1.581683          1.35089          0.531286         0.652446

   Proximity_to_Mountains  Proximity_to_Beaches      Pets  \
0                1.064292              1.270154  1.000787
1               -0.854741             -0.105311 -0.999214
2                1.642314             -0.521417  1.000787
3               -0.484807             -0.278688 -0.999214
4               -1.571488              1.489766  1.000787
```

	Environmental_Concerns	Gender_male	Gender_non-binary
0	0.996360	1.416644	-0.700178
1	-1.003653	1.416644	-0.700178
2	-1.003653	1.416644	-0.700178
3	-1.003653	1.416644	-0.700178
4	0.996360	-0.705894	-0.700178

	Preferred_Activities_skiing	Preferred_Activities_sunbathing
0	1.726646	-0.579561
1	-0.579158	1.725443
2	1.726646	-0.579561
3	-0.579158	-0.579561
4	-0.579158	1.725443

	Preferred_Activities_swimming	Location_suburban	Location_urban
0	-0.574277	-0.707183	-0.706462
1	-0.574277	1.414062	-0.706462
2	-0.574277	-0.707183	-0.706462
3	1.741320	-0.707183	-0.706462
4	-0.574277	1.414062	-0.706462

	Favorite_Season_spring	Favorite_Season_summer	Favorite_Season_winter
0	1.751731	-0.581506	-0.575635
1	-0.570864	1.719672	-0.575635
2	-0.570864	1.719672	-0.575635
3	-0.570864	1.719672	-0.575635
4	-0.570864	-0.581506	-0.575635

Scaled Testing Data:

	Age	Income	Education_Level	Travel_Frequency	Vacation_Budget
0	1.235244	-1.077332	0.45486	0.183639	0.087137
1	-1.100517	0.141526	1.35089	0.183639	0.013100
2	-1.500934	-0.270850	0.45486	-0.859303	-1.149139
3	0.834828	0.656139	0.45486	-0.859303	1.676635
4	1.035036	0.924157	-0.44117	0.183639	1.383569

	Proximity_to_Mountains	Proximity_to_Beaches	Pets
0	-0.854741	-0.209337	1.000787
1	0.786841	0.611318	1.000787
2	-0.947224	0.645994	1.000787
3	1.688555	1.593793	1.000787
4	-1.155312	-1.527009	-0.999214

	Environmental_Concerns	Gender_male	Gender_non-binary
0	0.996360	-0.705894	-0.700178
1	0.996360	-0.705894	1.428208
2	-1.003653	1.416644	-0.700178
3	-1.003653	-0.705894	-0.700178
4	0.996360	-0.705894	1.428208

```
   Preferred_Activities_skiing  Preferred_Activities_sunbathing  \
0                    -0.579158                        -0.579561
1                     1.726646                        -0.579561
2                     1.726646                        -0.579561
3                     1.726646                        -0.579561
4                    -0.579158                         1.725443

   Preferred_Activities_swimming  Location_suburban  Location_urban  \
0                       1.741320          -0.707183        1.415504
1                      -0.574277          -0.707183        1.415504
2                      -0.574277           1.414062       -0.706462
3                      -0.574277          -0.707183       -0.706462
4                      -0.574277           1.414062       -0.706462

   Favorite_Season_spring  Favorite_Season_summer  Favorite_Season_winter
0               -0.570864                1.719672               -0.575635
1               -0.570864               -0.581506                1.737213
2               -0.570864               -0.581506               -0.575635
3                1.751731               -0.581506               -0.575635
4               -0.570864               -0.581506               -0.575635
```

Step 1. Choose Three of The Following Classifiers to Use Them For The Prediction of Vacation Preferences:

The following three classifiers were chosen thoroughly for the reasons below:

Classifier 1: Decision Trees

1. Handling Mixed Data Types:
    - Our dataset has both numerical (Age, Income, Proximity_to_Mountains) and categorical features (Gender, Preferred_Activities, Location).
    - Decision Trees are a natural fit for such datasets since they process mixed data types directly without requiring transformations like one-hot encoding or scaling.
2. Feature Importance and Interpretability:
    - From the EDA, categorical features like Preferred_Activities and Location appear to have distinct distributions relative to Preference. Decision Trees provide feature importance insights, which align with the project's goal of understanding the influence of features on vacation preferences.

3. Robustness to Outliers:
    - Outliers were observed in features like Vacation_Budget and Proximity_to_Mountains. Decision Trees are unaffected by these because splits depend on thresholds, not statistical assumptions.
4. Non-Linearity:
    - Based on weak correlations between numerical features, linear models may struggle. Decision Trees excel at capturing non-linear decision boundaries.

Classifier 2: Neural Networks

1. Large Dataset Size:
    - Our dataset has 52,444 samples, providing sufficient data for Neural Networks to learn complex patterns. Neural Networks perform well when ample data is available, as is the case here.
2. Non-Linear Relationships:
    - Weak correlations among numerical features (e.g., between Income, Proximity_to_Beaches, and Vacation_Budget) suggest non-linear relationships that Neural Networks can model effectively.
3. Diverse Feature Set:
    - Features like Age, Income, Travel_Frequency, and Proximity_to_Beaches show varied distributions, which Neural Networks can process effectively by adjusting weights during training. They can learn intricate dependencies between features like Favorite_Season and Preference.
4. Slight Class Imbalance:
    - Neural Networks can handle slight class imbalances in Preference by adjusting loss functions or using techniques like oversampling (e.g., SMOTE) during preprocessing.

Classifier 3: Support Vector Machines (SVM)

1. Feature Scaling for Numerical Features:
    - Numerical features like Income, Vacation_Budget, and Proximity_to_Mountains have varying scales, as observed in the EDA. SVM requires proper scaling to

optimize performance, making it suitable for this dataset when combined with normalization techniques.

2. Handling Class Imbalance:
    - The slight imbalance in Preference (observed during class distribution analysis) can be addressed by SVM using class weights, ensuring fairness in classification.
3. Versatility with Kernels:
    - From EDA, weak correlations in numerical features suggest non-linearity in the data. SVM's kernel trick (e.g., radial basis function or polynomial kernel) is effective in capturing such relationships.
4. High-Dimensional Feature Space:
    - With multiple numerical and categorical features, SVM's ability to find hyperplanes in high-dimensional spaces is advantageous, especially if non-linear patterns dominate.

Why Other Classifiers Were Excluded Based on EDA:

1. Naïve Bayes:
    - Assumes independence between features, but EDA shows correlations and interactions between features like Proximity_to_Beaches and Preferred_Activities, which violate this assumption.
    - Limited utility for mixed data types without significant preprocessing.
2. K-Nearest Neighbors (KNN):
    - Sensitive to outliers like those in Vacation_Budget and Proximity_to_Mountains, which can skew nearest neighbor calculations.
    - Computationally expensive for large datasets like ours during prediction due to repeated distance calculations.

Summary of EDA-Based Justification

Classifier	Key Strengths Aligned with EDA Findings
Decision Trees	Handles mixed data, interpretable, robust to outliers, and captures non-linear patterns.

Classifier	Key Strengths Aligned with EDA Findings
Neural Networks	Leverages large dataset size, captures non-linear relationships, and processes diverse features effectively.
SVM	Effective for non-linear relationships, scales well with proper feature scaling, and handles class imbalance.

Models Implementation And Testing:

DECISION TREE:

```
Training Data Accuracy: 1.0

Classification Report for Training Data:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     31437
           1       1.00      1.00      1.00     10518

    accuracy                           1.00     41955
   macro avg       1.00      1.00      1.00     41955
weighted avg       1.00      1.00      1.00     41955

Confusion Matrix for Training Data:
[[31437     0]
 [    0 10518]]

Test Data Accuracy: 0.9958051291829536

Classification Report for Test Data:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      7859
           1       0.99      0.99      0.99      2630

    accuracy                           1.00     10489
   macro avg       0.99      0.99      0.99     10489
weighted avg       1.00      1.00      1.00     10489

Confusion Matrix for Test Data:
[[7840   19]
 [  25 2605]]
```

SVM:

```
Training Data Accuracy (SVM): 0.9966630914074603

Classification Report for Training Data (SVM):
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     31437
           1       0.99      0.99      0.99     10518

    accuracy                           1.00     41955
   macro avg       1.00      1.00      1.00     41955
weighted avg       1.00      1.00      1.00     41955

Confusion Matrix for Training Data (SVM):
[[31376    61]
 [   79 10439]]

Test Data Accuracy (SVM): 0.9890361330918105

Classification Report for Test Data (SVM):
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      7859
           1       0.98      0.98      0.98      2630

    accuracy                           0.99     10489
   macro avg       0.99      0.99      0.99     10489
weighted avg       0.99      0.99      0.99     10489

Confusion Matrix for Test Data (SVM):
[[7799   60]
 [  55 2575]]
```

NEURAL NETWORK:

```
Training Data Accuracy (Neural Network): 0.9998808246931236

Classification Report for Training Data (Neural Network):
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     31437
           1       1.00      1.00      1.00     10518

    accuracy                           1.00     41955
   macro avg       1.00      1.00      1.00     41955
weighted avg       1.00      1.00      1.00     41955

Confusion Matrix for Training Data (Neural Network):
[[31437     0]
 [    5 10513]]

Test Data Accuracy (Neural Network): 0.9968538468872151

Classification Report for Test Data (Neural Network):
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      7859
           1       1.00      0.99      0.99      2630

    accuracy                           1.00     10489
   macro avg       1.00      0.99      1.00     10489
weighted avg       1.00      1.00      1.00     10489

Confusion Matrix for Test Data (Neural Network):
[[7851    8]
 [  25 2605]]
```

Step 2. Assess the chosen three classifier in Step 1). The principle is to compare between the three classifiers and to use the concept of model validation seen in the assignment 2.

Since we have proceeded with Decision Tree, Neural Network, and SVM models. Each will be implemented and evaluated using appropriate model validation techniques learnt in Assignment 2.

For Decision Tree we got the below outputs:

1. K-Fold Cross-Validation Results for Decision Tree:
    - Mean Accuracy: 0.9952
    - Std Dev: 0.0010

2. Leave-One-Out Cross-Validation Results for Decision Tree:
    - Mean Accuracy: 0.9962

3. Grid Search Results for Decision Tree:
    - Best Parameters: {'max_depth': 15, 'min_samples_leaf': 1, 'min_samples_split': 5}
    - Best Score: 0.9954

4. Randomized Search Results for Decision Tree:
    - Best Parameters: {'max_depth': 10, 'min_samples_leaf': 3, 'min_samples_split': 12}
    - Best Score: 0.9950

5. Halving Grid Search Results for Decision Tree:
    - Best Parameters: {'max_depth': 15, 'min_samples_leaf': 2, 'min_samples_split': 2}
    - Best Score: 0.9954

For Neural Network we got the below outputs:

1. K-Fold Cross-Validation Results for Neural Network:
    - Mean Accuracy: 0.9975
    - Std Dev: 0.0006

2. Leave-One-Out Cross-Validation Results for SVM:
    - Mean Accuracy: 0.9958

3. Grid Search Results for Neural Network:

- Best Parameters: {'alpha': 0.0001, 'hidden_layer_sizes': (100,), 'learning_rate': 'constant'}
- Best Score: 0.9975

4. Randomized Search Results for Neural Network:
    - Best Parameters: {'alpha': 0.0006641157902710027, 'hidden_layer_sizes': (100,), 'learning_rate': 'adaptive'}
    - Best Score: 0.9974

5. Halving Grid Search Results for Neural Network:
    - Best Parameters: {'alpha': 0.0001, 'hidden_layer_sizes': (100,), 'learning_rate': 'constant'}
    - Best Score: 0.9975

For SVM we got the below outputs:

1. K-Fold Cross-Validation Results for SVM:
    - Mean Accuracy: 0.9983
    - Std Dev: 0.0001

2. Leave-One-Out Cross-Validation Results for SVM:
    - Mean Accuracy: 0.9720

3. Grid Search Results for SVM:
    - Best Parameters: {'C': 10, 'gamma': 'scale', 'kernel': 'linear'}
    - Best Score: 0.9992

4. Randomized Search Results for SVM:
    - Best Parameters: {'C': 6.1111501174320875, 'gamma': 'auto', 'kernel': 'linear'}
    - Best Score: 0.9993

5. Halving Grid Search Results for SVM:
    - Best Parameters: {'C': 10, 'gamma': 'scale', 'kernel': 'linear'}
    - Best Score: 0.9992

ANALYSIS OF OVERFITTING FOR EACH MODEL

1.Decision Tree:

- Training Accuracy: 1.0 (perfect accuracy is a strong indicator of overfitting).
- Test Accuracy: 0.9958 (slightly lower but still close to the training accuracy).
- Validation Results: Consistent with test accuracy, but the perfect training accuracy indicates the model may be memorizing the training data rather than generalizing.

Why Overfitting?

- Decision Trees tend to overfit when:
    - Unrestricted Depth: If max_depth is very large or set to None, the tree becomes too complex and fits the training data perfectly.
    - Overly Granular Splits: Splits on minor variations in the data create branches specific to the training set, reducing generalizability.
- Conclusion: The perfect training accuracy with a slight drop in test accuracy suggests mild overfitting.

2. Neural Network:

- Training Accuracy: 0.9998 (almost perfect, indicating a potential for overfitting).
- Test Accuracy: 0.9968 (close to training accuracy, suggesting good generalization).
- Validation Results: Consistent with test accuracy, meaning overfitting is minimal.

Why Overfitting is Less Pronounced?

- Neural Networks can overfit when:
    - Large Hidden Layers: High model complexity allows the network to memorize training data.
    - Insufficient Regularization: Without techniques like dropout or weight penalties (e.g., L2 regularization), the model can overfit.

- Conclusion: While the training accuracy is nearly perfect, the test accuracy is also high, suggesting only slight overfitting or potentially due to good generalization.

3. SVM:

- Training Accuracy: 0.9966 (not perfect, indicating less likelihood of overfitting).
- Test Accuracy: 0.989 (lower than training accuracy but still strong, showing good generalization).
- Validation Results: Consistent with test accuracy, with no significant gaps between training, testing, and validation.

Why Overfitting is Minimal?

- SVMs are inherently robust to overfitting due to:
    - Margin Maximization: SVMs find a decision boundary that maximizes the margin, improving generalization.
    - Tuning Parameter C: If C is very high, it can overfit by trying to perfectly classify the training data. However, the moderate training accuracy suggests appropriate regularization.
- Conclusion: The consistent performance across training, testing, and validation suggests minimal or no overfitting.

Overall Assessment:

- Decision Tree: Clear signs of overfitting due to perfect training accuracy and unrestricted complexity.
- Neural Network: Slight overfitting but acceptable due to strong generalization.
- SVM: No significant overfitting, as evidenced by consistent performance and lower training accuracy relative to Decision Tree and Neural Network.

ANALYSIS OF OVERFITTING IN DECISION TREE AFTER APPLYING TECHNIQUES IN ASSIGNMENT 2

Evaluation of whether overfitting improved:

1. K-Fold Cross-Validation:

- **Mean Accuracy**: 0.9952
- **Standard Deviation**: 0.0010
- **Observation**: The consistent high accuracy with low standard deviation across folds indicates that the model generalizes well to different subsets of the data. Overfitting seems to have been controlled.

2. Leave-One-Out Cross-Validation (LOOCV):

- **Mean Accuracy**: 0.9962
- **Observation**: The slightly higher accuracy reflects the model's ability to handle individual samples well. LOOCV being computationally intensive yet producing high accuracy further suggests that overfitting is minimal.

3. Grid Search Results:

- **Best Parameters**:
    - max_depth=15, min_samples_leaf=1, min_samples_split=5
- **Best Score**: 0.9954
- **Observation**: The parameter tuning effectively balanced the tree complexity, reducing overfitting. The chosen depth and split restrictions ensure the tree doesn't overfit while maintaining strong performance.

4. Randomized Search Results:

- **Best Parameters**:
    - max_depth=10, min_samples_leaf=3, min_samples_split=12
- **Best Score**: 0.9950

- **Observation**: A slightly lower score compared to Grid Search, likely due to stricter splits (min_samples_leaf=3). This suggests a trade-off between slight underfitting and reduced overfitting.

5. Halving Grid Search Results:

- **Best Parameters**:
  - max_depth=15, min_samples_leaf=2, min_samples_split=2
- **Best Score**: 0.9954
- **Observation**: Similar results to regular Grid Search, showing that hyperparameter tuning effectively mitigated overfitting while preserving high accuracy.

Overall Impact on Overfitting:

1. **Before Applying Techniques**:
   - The Decision Tree exhibited signs of overfitting, as indicated by **perfect training accuracy (1.0)** and slightly lower test accuracy (e.g., 0.9958).
2. **After Applying Techniques**:
   - Limiting max_depth and restricting splits (min_samples_split and min_samples_leaf) reduced the tree's tendency to memorize training data, leading to **consistent validation scores** across K-Fold, LOOCV, and Grid/Randomized Search.
   - The model generalizes better with no drastic performance drop across training, validation, and test datasets.
3. **Conclusion**:
   - The overfitting has been mitigated effectively. The model now balances complexity and generalization, as seen from consistent results across different validation techniques.
   - Randomized Search results indicate further opportunities to test stricter parameters (e.g., smaller tree depths or more restrictive splits) for optimal generalization.

ANALYSIS OF OVERFITTING IN NEURAL NETWORK AFTER APPLYING TECHNIQUES

Before Applying Techniques:

1. **Training Data Performance**:
    - **Accuracy**: 0.9998 (~100%), indicating the model nearly memorized the training data.
    - **F1-Score**: 1.00 for both classes, showing perfect performance.
2. **Test Data Performance**:
    - **Accuracy**: 0.9968 (~99.7%), suggesting good generalization with minimal overfitting.
    - **F1-Score**: 1.00 for class 0 and 0.99 for class 1, indicating strong performance.

Key Observation:

- The nearly perfect training accuracy suggests slight overfitting, as the model may be over-relying on the training data patterns. However, the high-test accuracy indicates that overfitting is not yet severe.

After Applying Techniques:

1. **K-Fold Cross-Validation**:
    - **Mean Accuracy**: 0.9975
    - **Standard Deviation**: 0.0006 (very low, indicating consistent performance across folds).
    - **Observation**: Cross-validation ensures better generalization, and the high mean accuracy with low variability confirms minimal overfitting.
2. **Leave-One-Out Cross-Validation (LOOCV)**:
    - **Mean Accuracy**: 0.9958
    - **Observation**: LOOCV results align closely with K-Fold, indicating robust performance on individual samples.
3. **Grid Search Results**:

- **Best Parameters**:
    - alpha: 0.0001 (regularization parameter, mitigating overfitting).
    - hidden_layer_sizes: (100,) (moderate complexity, avoiding excessive hidden units).
    - learning_rate: 'constant' (stable learning rate for training).
- **Best Score**: 0.9975

4. **Randomized Search Results**:
    - **Best Parameters**:
        - alpha: 0.0006641157902710027 (slightly stronger regularization).
        - hidden_layer_sizes: (100,) (same as Grid Search).
        - learning_rate: 'adaptive' (adjusts learning rate for optimal convergence).
    - **Best Score**: 0.9974
5. **Halving Grid Search Results**:
    - **Best Parameters**: Same as Grid Search results.
    - **Best Score**: 0.9975

Comparison of Overfitting (Before vs. After)

Metric	**Before Techniques**	**After Techniques**	**Analysis**
Training Accuracy	0.9998	0.9975 (via CV)	Slight drop in training accuracy indicates reduced overfitting.
Test Accuracy	0.9968	0.9975 (via Grid Search)	Slight improvement due to better generalization.
Cross-Validation Scores	Not applied	Mean: 0.9975, Std Dev: 0.0006	Consistent scores confirm improved robustness.
Regularization (alpha)	Not applied	Applied (0.0001)	Regularization prevents overfitting by penalizing complexity.

Metric	Before Techniques	After Techniques	Analysis
Hidden Layers	Default	Reduced to (100,)	Simplified architecture helps avoid memorization.

Conclusion:

1. **Overfitting Improved**:
    - The application of regularization, simplified hidden layers, and hyperparameter tuning effectively reduced overfitting.
    - Training accuracy is no longer near-perfect, showing the model has shifted from memorizing to learning generalizable patterns.
2. **Generalization Enhanced**:
    - Consistent performance across K-Fold, LOOCV, and test data indicates the model generalizes well to unseen data.
3. **Key Takeaways**:
    - The use of **alpha (regularization)** and reduced architecture was crucial in balancing training and test performance.
    - Model validation techniques like K-Fold and Grid Search provided reliable performance metrics and hyperparameter optimization.

ANALYSIS OF OVERFITTING IN SVM AFTER APPLYING TECHNIQUES

Before Applying Techniques:

1. **Training Data Performance**:
    - **Accuracy**: 0.9967 (~99.67%), showing strong training performance with slight potential overfitting.
    - **F1-Score**: 1.00 for class 0 and 0.99 for class 1, indicating near-perfect classification.
2. **Test Data Performance**:
    - **Accuracy**: 0.989 (~98.9%), showing good generalization.

- **F1-Score**: 0.99 for both classes.

Observation:

- The model exhibits excellent performance but shows a small gap between training and test accuracy, indicating mild overfitting.

After Applying Techniques:

1. **K-Fold Cross-Validation**:
    - **Mean Accuracy**: 0.9983
    - **Std Dev**: 0.0001 (extremely low, indicating highly consistent performance).
2. **Leave-One-Out Cross-Validation**:
    - **Mean Accuracy**: 0.9720 (slightly lower, as expected for LOOCV due to its sensitivity to individual data points).
3. **Grid Search Results**:
    - **Best Parameters**: {'C': 10, 'gamma': 'scale', 'kernel': 'linear'}
    - **Best Score**: 0.9992
4. **Randomized Search Results**:
    - **Best Parameters**: {'C': 6.11, 'gamma': 'auto', 'kernel': 'linear'}
    - **Best Score**: 0.9993
5. **Halving Grid Search Results**:
    - **Best Parameters**: Same as Grid Search.
    - **Best Score**: 0.9992

Comparison of Overfitting (Before vs. After)

Metric	**Before Techniques**	**After Techniques**	**Analysis**
Training Accuracy	0.9967	~0.9992 (via Grid Search)	Slight improvement due to better tuning but no significant overfitting.

Metric	Before Techniques	After Techniques	Analysis
Test Accuracy	0.989	~0.9983 (via CV)	Improved generalization with reduced performance gap.
Cross-Validation Scores	Not applied	Mean: 0.9983, Std Dev: 0.0001	Highly consistent scores confirm improved robustness.
Hyperparameter Tuning	Default	C, gamma, and kernel tuned	Regularization and kernel selection reduced overfitting.

Conclusion:

1. **Overfitting Reduced**:
    - After applying techniques (e.g., tuning C, gamma, and kernel), the training and test accuracy gap narrowed, indicating improved generalization.
2. **Generalization Enhanced**:
    - Consistent results from cross-validation and hyperparameter optimization confirm the model is better balanced and less prone to overfitting.
3. **Key Takeaways**:
    - Regularization (C) and appropriate kernel selection (linear) played a crucial role in reducing overfitting while maintaining excellent performance.

Step 3. Possibility to increase the scores. You have to check the possibility to increase the obtained scores in Step 2. Innovation in the experiment to improve the obtained scores will be an asset.

The following details highlight our efforts and strategies to enhance model performance and improve scoring metrics.

1. **Addressing Class Imbalance**

Class imbalance posed a challenge, with one class significantly outnumbering the other. To counter this, we employed the **Synthetic Minority Oversampling Technique (SMOTE)**.

SMOTE Application:

- **Training Set Balancing**:
    - SMOTE was applied exclusively to the training dataset after splitting the data into training and testing subsets.
    - Synthetic samples were generated for the minority class by interpolating between existing samples, ensuring equal representation of both classes in the training data.
- **Validation of Effectiveness**:
    - The class distribution was visualized before and after applying SMOTE, confirming a balanced training set.
    - The balanced distribution successfully mitigated bias, preparing the dataset for robust modeling.

2. **Scaling the Features**

To optimize model performance and enhance score reliability, we evaluated feature scaling methods, **Normalization** and **Standardization**, after addressing class imbalance with SMOTE.

Implementation Details:

- **Decision to Use Standardization**:
    - Both scaling techniques were tested, and **Standardization** using StandardScaler was selected due to its superior performance with our dataset and models.
- **Training and Test Set Scaling**:
    - The scaler was **fitted on the training dataset** (X_train_balanced) and used to transform the training data. This prevented data leakage and ensured a robust model training process.
    - The test set (X_test) was transformed using the **same fitted scaler**, maintaining test set integrity and preserving unseen data characteristics.

Exploring PCA with Scaled Data:

- **Experimentation**:
    - Principal Component Analysis (PCA) was applied to the standardized data to reduce dimensionality and potentially improve model performance.

- The results of PCA-transformed datasets were analyzed and documented in the notebook for reference.

3. Dimensionality Reduction with PCA

PCA was implemented to simplify the dataset, reduce computational complexity, and assess its impact on model performance.

Implementation Details:

- **Application of PCA**:
  - PCA was applied after scaling to prevent feature magnitude from biasing the computation of principal components.
  - The number of components was chosen to retain **95% of the variance**, balancing dimensionality reduction with information retention.
- **Consistency Across Datasets**:
  - PCA was **fitted on the scaled training data** (X_train_balanced_scaled) and the same transformation was applied to the test set (X_test_scaled), ensuring compatibility.
- **Variable Reassignment**:
  - The PCA-transformed datasets were reassigned to the original variable names (X_train and X_test) for consistency in the pipeline and seamless integration with subsequent modeling steps.

Note: After the implementing the project on the encoded dataset, the entire project was re-implemented using encoded PCA-transformed data. While the re-implemented results are not documented in this report, they are fully detailed in the submitted google colab notebook, with each step clearly demonstrated.

CONCLUSION

This project successfully explored and implemented various machine learning techniques to predict vacation preferences based on diverse features. The following achievements, drawbacks, and opportunities for improvement summarize the outcomes of the work:

Achievements:

1. Class Imbalance Addressed:
    - The use of SMOTE effectively balanced the training dataset, mitigating biases and improving the robustness of model training.
    - Visualization of class distribution before and after SMOTE confirmed the success of this approach.
2. Feature Scaling and Dimensionality Reduction:
    - StandardScaler provided a robust scaling mechanism, preserving the integrity of the test set and ensuring optimal feature transformation.
    - PCA reduced the dataset's dimensionality, retaining 95% of the variance, which simplified the data and maintained information integrity.
3. Model Selection and Evaluation:
    - Decision Tree, Neural Network, and SVM were carefully selected based on dataset characteristics and evaluated using advanced validation techniques (e.g., K-Fold, LOOCV, Grid Search).
    - Overfitting was addressed and minimized across models, with SVM achieving the most consistent generalization and Neural Network showing improved robustness post-regularization.
4. Model Performance:
    - All three models demonstrated high accuracy, with SVM achieving the highest validation score, closely followed by Neural Network and Decision Tree.

Drawbacks:

1. Decision Tree Overfitting:
    - Despite hyperparameter tuning, Decision Tree exhibited signs of overfitting due to its inherent tendency to memorize training data when the depth is unrestricted.

2. Computational Complexity:
    - Neural Networks and SVM required extensive computation during hyperparameter tuning, particularly with large datasets and complex kernels.
3. Limited Exploration of Alternate Models:
    - While three classifiers were chosen based on dataset characteristics, models like ensemble methods (e.g., Random Forest, Gradient Boosting) might have offered further insights.

Opportunities for Improvement:

1. Fine-Tuning Parameters:
    - Further optimization of hyperparameters, especially for Decision Tree and Neural Network, could enhance generalization and mitigate minor overfitting issues.
2. Incorporating Additional Features:
    - Expanding the feature set or deriving new features from existing ones could improve model accuracy and insights into vacation preferences.
3. Alternative Techniques:
    - Exploring ensemble learning, hybrid models, or feature selection techniques could further improve results and reduce computational overhead.
4. Validation on Unseen Data:
    - Testing the models on a completely independent dataset would provide more robust validation of the model's generalization capabilities.

In conclusion, the project achieved its primary objective of building effective models for predicting vacation preferences, with strong performance metrics across all three classifiers. While there are opportunities for refinement, the insights gained and methodologies implemented lay a solid foundation for future work and further exploration.